

Infrastructure-as-Code MLOps for Predictive Analytics

Anonymous Author(s)

Affiliations withheld for double-blind review

Abstract. Institutional predictive systems are hard to reproduce once ingestion, evaluation, deployment, and reporting are built as separate steps. We present an Infrastructure-as-Code Machine Learning Operations (MLOps) workflow that binds those steps into one traceable pipeline: a deterministic adapter feeds PostgreSQL persistence, a blocked temporal evaluator chooses a forecasting model, and a schema-validated reporter emits auditable evidence, all run in containers and declared for Azure through Terraform. We evaluated it on a dataset of 87,073 administrative publication records from 2015–2023, aggregated into nine annual observations. Over three chronological folds, eight candidates were compared by mean absolute percentage error (MAPE) and fold instability; Naive and simple exponential smoothing tied for the best score, and parsimony kept Naive, which projected 15,229 records per year for 2024–2026 within an exploratory residual-calibrated 80% interval of 9,541.5–20,916.5. Host and container runs agreed on every non-numeric field and, on the 45 numeric differences, stayed inside a declared absolute-or-relative tolerance, even though their full payload hashes diverged. The result is an auditable evidence contract that ties source identity, model-selection evidence, generated artifacts, and reporting outputs to a single reproducible execution.

Keywords: reproducibility · auditable evidence · data provenance · temporal cross-validation · containerized workflows

1 Introduction

Machine learning (ML) systems now inform forecasting and institutional decisions [1], yet a prediction is only as trustworthy as the trail that documents how it was produced [2]. Machine Learning Operations (MLOps) builds that trail by drawing data ingestion, model development, deployment, monitoring, and governance into a single traceable lifecycle whose reliability in production is itself an active concern [3–6]. Reproducibility then rests on keeping four things intact: the identity of the dataset, the conditions under which it ran, the evidence behind model selection, and the artifacts produced along the way.

In practice these stages are often built and run in isolation. When they drift apart, the datasets, metrics, figures, and outputs produced in one environment stop matching those produced in another. Prior taxonomies and data-pipeline

studies map the lifecycle and document how schema, quality, and provenance failures propagate downstream [7–9], and data-centric perspectives treat data quality and governance as first-class pipeline concerns rather than afterthoughts [10].

Containerized and provenance-aware workflows have been proposed to preserve dependencies, runtime conditions, and processing records [7, 8, 11, 12]; recent work captures end-to-end provenance across an ML pipeline [13], and frameworks now support reproducible computational experiments across scientific domains [14]. Yet most of these efforts either target one platform capability or describe the lifecycle in the abstract, and what is still missing is a single contract that binds the identity of the data, the preprocessing decisions, the temporal evaluation, the artifacts these yield, and the comparison of results across environments.

This paper designs and evaluates such a contract. Built as Infrastructure-as-Code, the workflow runs the whole path from a deterministic dataset adapter to auditable evidence generation inside containers, and declares its Azure counterpart through Terraform. Three research questions (RQs) frame the evaluation: do provenance records and scientific artifacts stay internally consistent (RQ1); are host and container executions numerically equivalent under a declared policy (RQ2); and which forecasting candidate is defensible for a nine-observation annual series under blocked temporal validation (RQ3).

The case rests on 87,073 administrative publication records from 2015–2023, collapsed into nine annual observations, so the study speaks to traceability, evidence consistency, and tolerance-based reproducibility within a bounded setting rather than to forecasting accuracy at large. We deliberately set aside any claim of a new algorithm, a production-grade platform, continuous monitoring, or superior Azure performance. Figure 1 traces the workflow from end to end.

2 Methods

We built the artifact under design-science guidance, keeping its construction, evaluation, and communication transparent [15, 16]. The work moved through the familiar design-science stages: naming the problem, fixing the solution objectives, designing and building the artifact, demonstrating it, and evaluating what it produced.

2.1 Architecture, Ingestion, and Evidence Contract

Six logical components make up the workflow: a dataset adapter, PostgreSQL persistence, a temporal evaluation engine, an evidence registry, a Streamlit interface, and a deterministic reporter [7, 8]. Docker Compose runs the version we actually evaluated, and because containers freeze their runtime dependencies and execution records, they carry most of the reproducibility guarantee [11, 12]. Terraform additionally declares the Azure side (Container Apps, networking, and a PostgreSQL Flexible Server), but the evidence we kept covers configuration alone, so we make no claim about how the workflow executes, matches, or

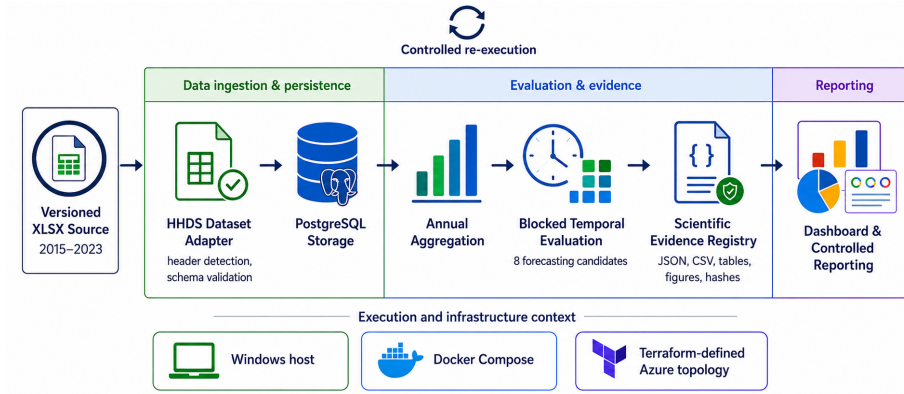


Fig. 1. End-to-end Infrastructure-as-Code MLOps workflow with three distinct stages marked by color progression. Green denotes the data ingestion and persistence stage, where raw data flows through deterministic adaptation into PostgreSQL storage. Blue represents the temporal evaluation and evidence generation stage, where multiple forecasting candidates are compared and their predictions registered for transparency. Violet indicates the deterministic reporting stage, where the narrative, tables, and figures are rendered directly from the frozen evidence registry to produce auditable outputs. The circular arrow illustrates the full, repeatable re-execution cycle, where identical code and pinned dependencies enable reproducibility across both host and container environments.

performs on Azure [17, 18]. Figure 2 sets these logical components against their local execution and their declared Azure counterparts.

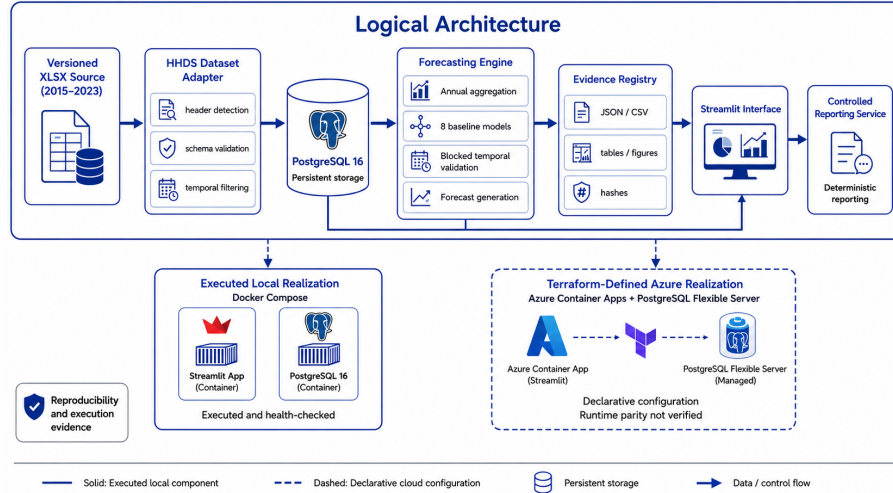


Fig. 2. Logical architecture and physical realizations. Solid borders denote executed local components; dashed borders denote Terraform-defined Azure resources whose runtime parity was not evaluated.

We drew the workbook from the official SENESCYT open-data repository [19]. A deterministic adapter, HHDS (Heuristic Header Detection and Sanitization), prepares it in a fixed sequence: it scans the first 20 rows, keeps the candidate header whose overlap with the declared schema is highest, drops empty or undeclared fields, checks that every year falls within 2015 to 2023, normalizes the categorical values, records provenance, and writes the clean table into PostgreSQL 16 [9, 20]. The retained fields span four dimensions: temporal, institutional, publication, and disciplinary. Figure 3 lays out this pipeline step by step.

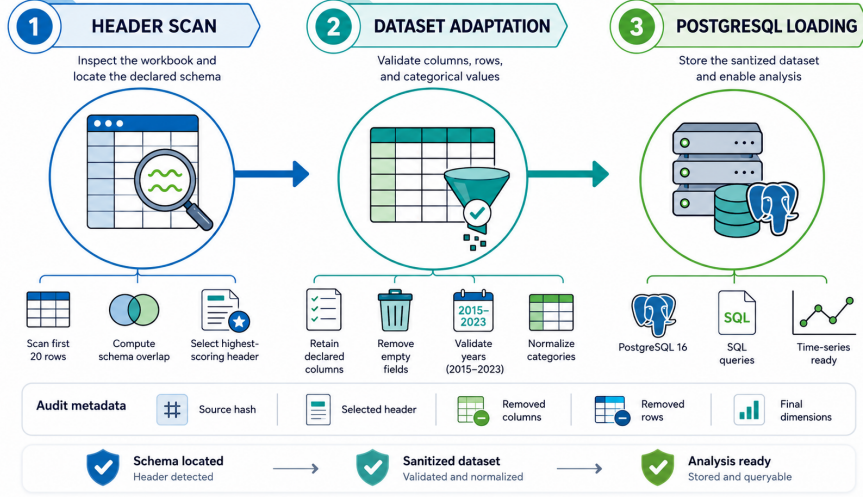


Fig. 3. Deterministic HHDS adaptation: header detection, schema validation, temporal filtering, normalization, provenance capture, and PostgreSQL loading.

Algorithm 1 states this procedure precisely, from header detection through sanitization to the registration of audit metadata.

Algorithm 1 Deterministic Header Detection and Dataset Sanitization

Require: Workbook D , declared schema C^* , scan depth $N = 20$, valid years [2015, 2023]

Ensure: Sanitized dataset D_s and audit metadata

- 1: **for** $i = 1$ to N **do**
 - 2: Normalize candidate row r_i and compute its overlap with C^*
 - 3: **end for**
 - 4: Select the earliest row among those with maximum schema overlap as the header
 - 5: Remove preceding rows, unnamed columns, and empty rows
 - 6: Retain declared fields and validate the year attribute
 - 7: Normalize categorical values and enforce data types
 - 8: Record source identity, selected header, removed fields, and row counts
 - 9: Load D_s into PostgreSQL 16
 - 10: **return** D_s and the registered audit metadata
-

Every analytical stage writes to one shared evidence registry, whose contract fixes what must be recorded: the source identity and schema decisions, the fold-

level predictions and the selected model, the forecasts and their intervals, and the generated artifacts together with environment metadata, section hashes, and the full scientific payload. Because each value is written once and later read from the same place, the figures, tables, and narrative can no longer disagree with the numbers behind them. Figure 4 follows this evidence from generation through registration to its integrity check.

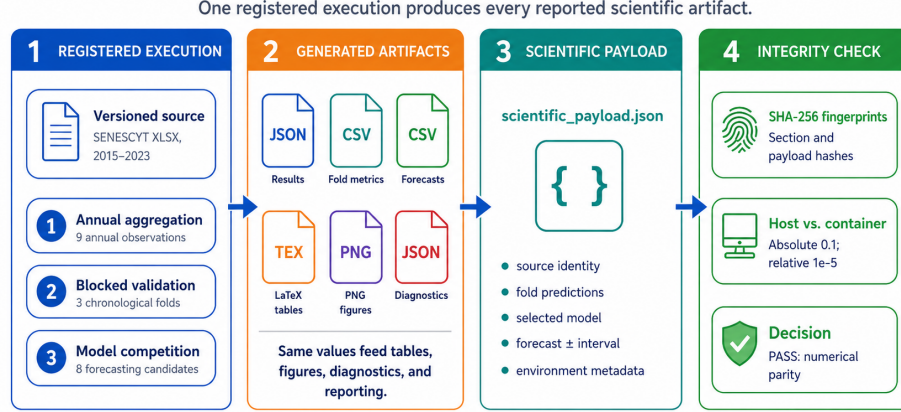


Fig. 4. Auditable evidence generation and integrity checking. Numerical equivalence uses the declared absolute-or-relative rule, while complete hashes preserve the distinction between numerical reproducibility and byte-level identity.

2.2 Temporal Evaluation and Forecasting

The target is the annual publication count. We compared eight candidates over three blocked folds that keep the years in chronological order [21–23]: Naive, simple exponential smoothing (SES), an automated exponential-smoothing state-space model (ETS), a linear trend, drift, the historical mean, a log-linear trend, and a Poisson generalized linear model (GLM). Equation (1) gives the fold-level mean absolute percentage error (MAPE) together with the instability-adjusted score that drives model selection.

$$\text{MAPE}_{m,k} = \frac{100}{n_k} \sum_{t \in V_k} \left| \frac{y_t - \hat{y}_{m,t}}{y_t} \right|, \quad S_\lambda(m) = \overline{\text{MAPE}}_m + \lambda s_m, \quad (1)$$

where s_m is the sample standard deviation of the fold MAPE. The main ranking set $\lambda = 1$, and we rechecked it at $\lambda \in \{0, 0.5, 1, 2\}$; ties were broken by parsimony, which is a selection rule rather than a claim of statistically significant superiority.

For the chosen model, six absolute out-of-fold residuals define an exploratory, residual-calibrated interval, written in Eq. (2).

$$PI_{h,0.8} = [\max(0, \hat{y}_h - q_{0.8}), \hat{y}_h + q_{0.8}], \quad (2)$$

where $q_{0.8}$ is the empirical residual quantile. The idea follows conformal prediction, though a calibration set this small and this dependent cannot promise population-level coverage [24, 25]. Figure 5 brings the validation, the candidate comparison, the instability-adjusted ranking, and the final choice of Naive into one view.

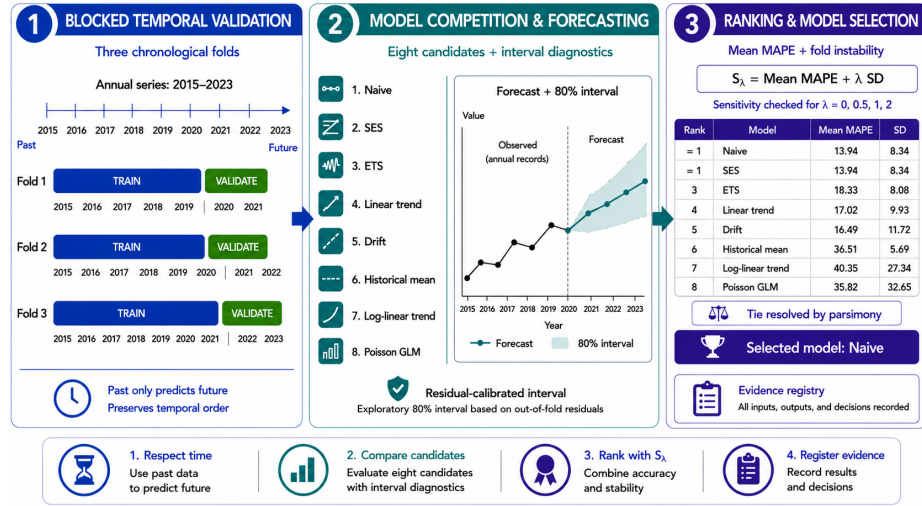


Fig. 5. Blocked temporal validation, comparison of eight forecasting candidates, instability-adjusted ranking, and selection of Naive by parsimony.

Algorithm 2 sets out the full computation, from temporal evaluation and forecasting to the registration of evidence.

Algorithm 2 Blocked Temporal Evaluation and Evidence Registration

Require: Annual series Y , candidate set $\mathcal{M}$, three chronological folds, horizon $H = \{2024, 2025, 2026\}$

Ensure: Selected model, forecasts, interval, and registered scientific payload

- 1: **for** each candidate $m \in \mathcal{M}$ **do**
 - 2: **for** each chronological fold k **do**
 - 3: Fit m using only observations preceding validation block V_k
 - 4: Predict V_k and compute $\text{MAPE}_{m,k}$
 - 5: Store fold predictions and absolute residuals
 - 6: **end for**
 - 7: Compute mean MAPE, fold SD, and the primary score $S_1(m)$
 - 8: **end for**
 - 9: Select the minimum-score candidate and resolve exact ties by parsimony
 - 10: Refit the selected model using the complete annual series
 - 11: Generate forecasts for H and the exploratory residual-calibrated interval
 - 12: Register folds, metrics, forecasts, figures, diagnostics, and hashes
 - 13: **return** the registered scientific payload
-

2.3 Reporting and Reproducibility Protocol

The reporting path we evaluated is deterministic and offline: a schema-validated template renders the narrative, tables, and figures straight from the registered payload. A GPT-4o-mini route exists as an option but was left switched off, so we say nothing here about the factuality, usefulness, or stability of large language models [26].

We compared host and container runs with complete and section-level cryptographic (SHA-256) fingerprints to detect any byte-level divergence in outputs. Non-numeric values had to match character for character. Two numeric values a and b counted as equal when they satisfied either branch of Eq. (3); the branches are joined by a logical OR, never an AND.

$$|a - b| \leq 0.1 \quad \vee \quad \frac{|a - b|}{\max(|a|, |b|)} \leq 10^{-5}. \quad (3)$$

In practice, then, an absolute difference of at most 0.1 or a relative difference of at most 10^{-5} was enough to treat two outputs as equal. The Linux/WSL2 container exposed eight logical CPUs and about 15.52 GB of memory, with no GPU. Host and container ran Python 3.14.5 and 3.14.6, so the test measures tolerance-based numerical reproducibility, not bitwise reproducibility under an identical stack.

3 Results

Every result below comes from one frozen execution and its containerized replica. The same pipeline code processed the same SENESCYT workbook on the Windows host and inside the Linux/WSL2 container, and the two environments differ only in operating system and Python patch level. Any divergence between them is therefore attributable to the runtime stack rather than to the inputs, the code path, or the selected model, which is what turns the host–container comparison into a controlled test of numerical reproducibility instead of a test of implementation differences. The host run is the reference, and the numbers, fingerprints, and intervals reported here are the values its evidence registry recorded, not quantities recomputed for presentation.

3.1 Execution and Cloud Configuration Evidence

Ingestion processed the 87,073 records and distinguished 16,599 raw journal-title strings in 16.44 s, about 5,296 records per second for this one workbook on this one machine; this is a single measurement, not a scalability figure. Figure 6 records this extract–transform–load (ETL) run.

The dashboard certifies the deterministic gates behind the counts: header detection, removal of unnamed columns and empty rows, and column-name normalization with year validation. The 87,073 records are therefore a post-sanitization figure with a recorded lineage, not a raw tally, and thus the fixed target the host

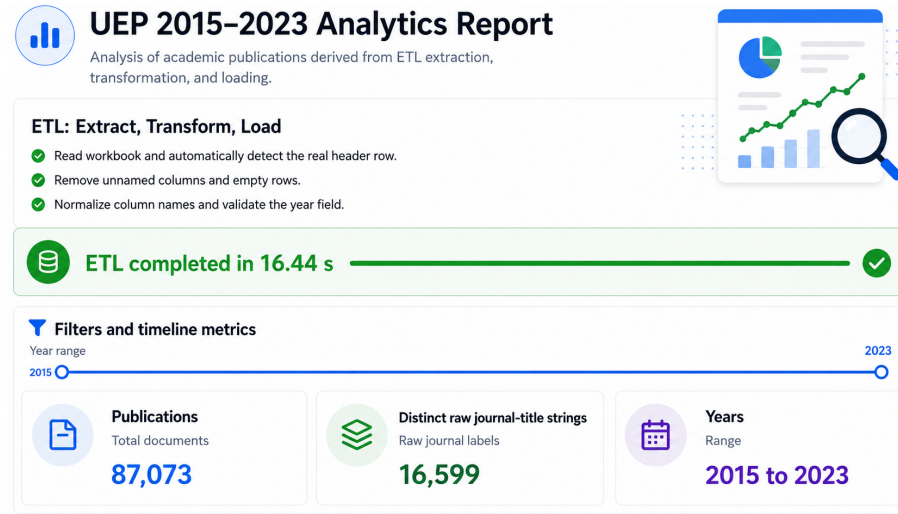


Fig. 6. Extract–transform–load run: 87,073 records and 16,599 distinct raw journal-title strings processed in 16.44 s.

and container must later reproduce. The 16,599 distinct raw journal-title strings measure the string-level heterogeneity the adapter absorbs, which is why publications are later grouped under normalized labels and why this summary anchors the provenance of the evidence contract rather than reporting performance.

Terraform declares the Azure application, its container environment, the networking, and a managed PostgreSQL instance. This evidence proves that the topology exists as declared, but it stops there: we captured no deployment transcript, health response, cloud payload, successful remote query, timing, or host–Azure comparison. Figure 7 shows the declared topology.

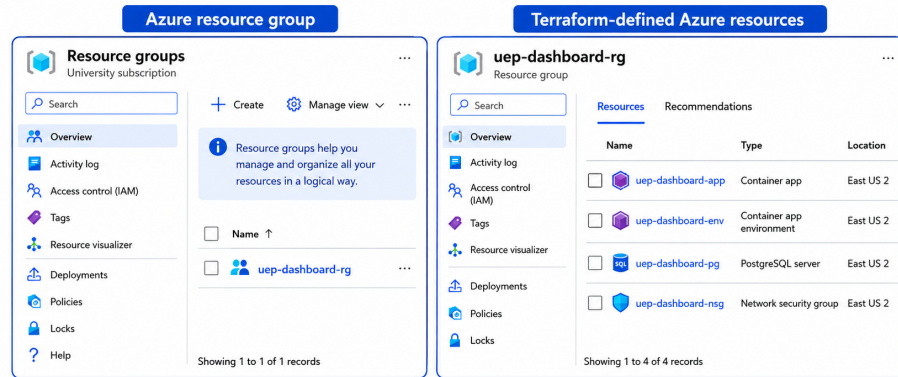


Fig. 7. Terraform-defined Azure resource topology. The figure documents declarative infrastructure, not cloud runtime execution or parity.

The resource group makes the local-to-cloud correspondence concrete: its four Terraform-declared members (a Container App, a Container App Environment, a PostgreSQL server, and a Network Security Group) mirror the runtime,

database, and network of the evaluated Docker Compose stack. Because they are enumerated from Terraform state rather than created by hand, the topology is reproducible by declaration, since re-applying the same code yields the identical named resources; this is exactly the configuration parity the paper claims and the sense in which the cloud target is auditable without being executed. Yet the claim is bounded: the group certifies a declared topology and network boundary, not a running application, so the security posture shown here is specified but its runtime enforcement remains untested.

3.2 Host–Container Reproducibility

The host ran Windows 11 with Python 3.14.5 and the container ran Linux/Windows Subsystem for Linux 2 (WSL2) with Python 3.14.6. Their full payload hashes came out different, but the cause was narrow: 45 small floating-point differences inside the forecasting section, with every non-numeric field identical. Table 1 gathers this evidence.

Table 1. Host–container reproducibility evidence.

Evidence	Recorded result
Payload SHA-256	Host: <code>ed7198a8bf5d...</code> Container: <code>a072859cc944...</code>
Exact parity	No; complete payload fingerprints differ
Numerical comparison	45 differences; max. absolute 0.0710614; max. relative 3.9943×10^{-5}
Policy and outcome	$ a - b \leq 0.1$ OR relative difference $\leq 10^{-5}$; passed. Cases exceeding the relative threshold satisfied the absolute branch
Scientific output	Naive selected in both; forecasts and intervals agreed under the declared rule

The reproducibility we can claim, then, is numerical and policy-bound: not byte-level identity, and not Azure runtime parity.

3.3 Descriptive and Forecasting Results

Annual counts climbed from 3,936 in 2015 to 15,229 in 2023 (3,936, 5,485, 8,611, 9,103, 9,518, 10,326, 11,088, 13,777, and 15,229), though these are administrative records rather than a direct gauge of scientific output. Figure 8 shows the leading journal labels under English display names, while the counts stay tied to the original raw strings.

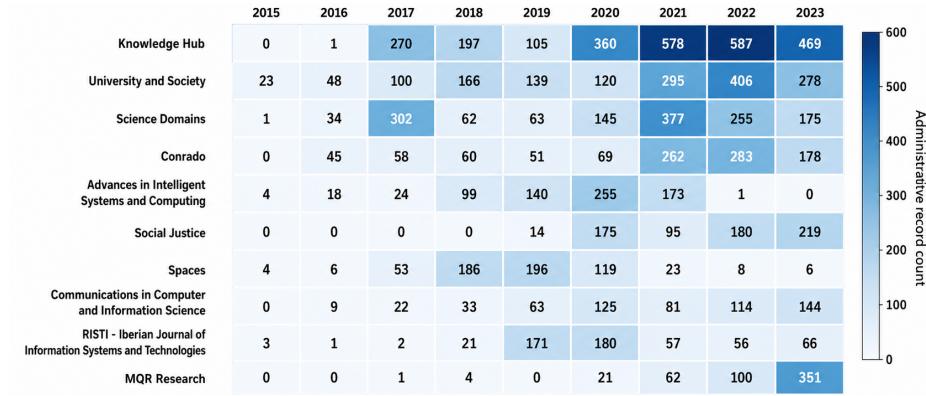


Fig. 8. The ten journal labels with the highest record counts over 2015–2023. Display names are English translations; the counts follow the original raw strings.

Naive and SES finished level, at mean MAPE 13.94, fold standard deviation 8.34, and $S_1 = 22.28$, so parsimony decided the tie in favor of Naive. The Poisson GLM looked strong in the last fold, at 3.5% MAPE, but swung too widely across the earlier folds to be trusted, and varying λ never broke the Naive–SES tie.

Naive therefore carries the 2023 level forward, projecting 15,229 records for each of 2024, 2025, and 2026, within an exploratory 80% interval of 9,541.5–20,916.5 (a residual radius of 5,687.5 records). Figure 9 is drawn straight from the registered forecast entries, not recomputed in the reporting layer.

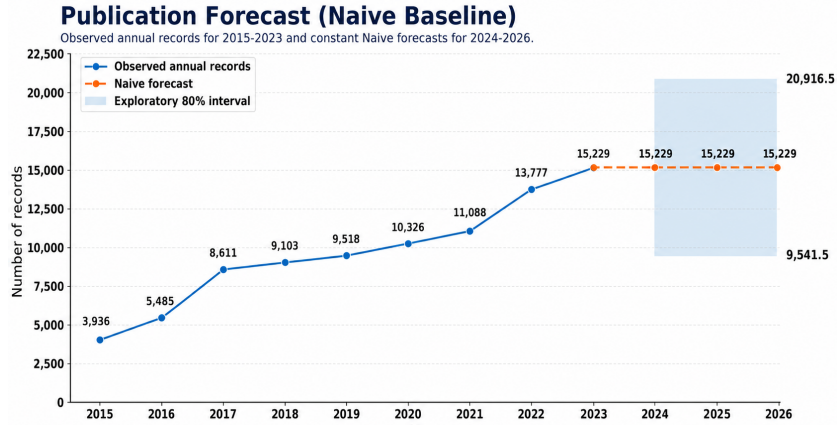


Fig. 9. Observed records and Naive forecasts with an exploratory 80% interval.

3.4 Deterministic Reporting

The deterministic reporter reused the same registered forecast, interval, and diagnostics as every other artifact, so the report cannot drift from the analysis behind it. The GPT-4o-mini route was configured but never run, which is why no language-model result appears here. Figure 10 lays out this reporting path.

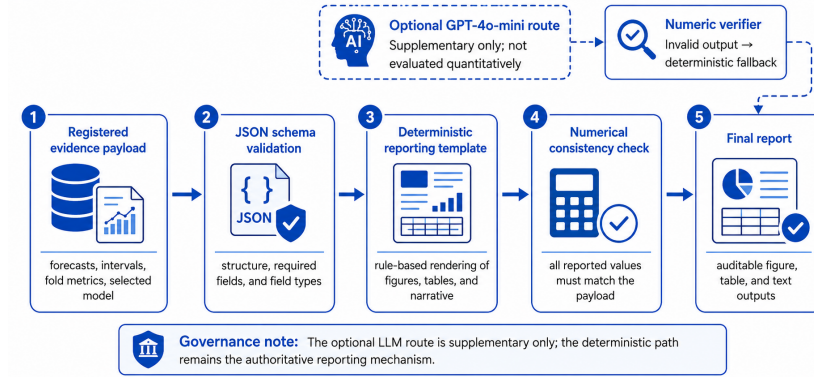


Fig. 10. Reporting workflow. The deterministic route is authoritative; the optional GPT-4o-mini route stays supplementary and unevaluated.

4 Discussion

Within this bounded case, the three questions have clear answers. The shared registry and its consistent artifacts settle RQ1; the exact match of non-numeric fields, together with the tolerance-level agreement of the 45 numeric differences, settles RQ2; and the exact Naïve–SES tie, broken by parsimony, answers RQ3. Where MLOps taxonomies and reviews chart the lifecycle and inventory its components, tools, and stakeholders [4, 7, 8, 20], and data-centric accounts push data quality and governance to the foreground [10], they largely stop at prescribing which stages should exist. Our contribution is complementary rather than competing: instead of adding another component to that map, the evidence contract binds the stages that already exist, so that identity, preprocessing, fold predictions, metrics, forecasts, figures, reports, and hashes all trace back to one registered execution and cannot quietly contradict one another.

This places the work alongside the container- and provenance-based reproducibility literature [11, 12], but with a different emphasis. Containers preserve the software environment [11] and provenance-aware engines capture execution records [12], with recent systems extending capture to end-to-end pipeline provenance [13]; our host–container comparison shows that this preservation is numerical rather than bitwise, since near-identical stacks under Python 3.14.5 and 3.14.6 still produced 45 small floating-point differences. That distance between code availability and identical results is the barrier that surveys of ML reproducibility document [2] and that cross-domain reproducibility frameworks set out to close [14]. Consistent with data-pipeline quality studies that trace downstream errors to schema and provenance failures [9], the deterministic adapter and the registry are exactly the points where such failures are caught before they reach the manuscript. The declarative-only treatment of Azure keeps this claim honest: unlike cloud-scaling and platform studies that examine runtime behavior [17, 18], we claim configuration parity, not execution parity.

On the forecasting side, the tie between simple baselines and more parameterized models is what the small-sample and temporal-validation literature would

anticipate. With nine annual points and three blocked folds, blocked temporal validation is the defensible protocol [21, 22], and results on model selection under temporal distribution shift caution against reading a single strong fold as durable evidence [23]; the Poisson GLM, strong in the final fold yet unstable earlier, is a clear instance. The exploratory interval follows conformal principles [24, 25], but we deliberately stop short of a coverage guarantee because the calibration set is small and non-exchangeable, precisely the regime those works flag. Simple baselines are therefore more defensible than parameterized rivals here, and the wide interval is the honest price of a short series rather than a defect of the workflow.

Reproducibility in our setting holds under an explicit tolerance rather than as exact binary identity, and the mismatched Python patch versions are a self-imposed limitation that a stricter replication should remove. Read against work on trustworthy and well-governed AI pipelines [26, 6, 1], the evidence contract operationalizes a narrow but auditable slice of that agenda: it records what was run and lets a third party recompute it, yet it does not yet address continuous training, drift, reliability, security, cost, or cloud scalability.

Construct validity is bounded by the outcome variable, since an administrative record count is a proxy for scientific productivity rather than a direct measure. Statistical-conclusion validity rests on nine observations, three folds, and six calibration residuals, so neither the ranking gaps nor the interval’s coverage should travel beyond this case. Internal validity hinges on the frozen adapter, the fold definitions, the pinned dependencies, and the registry, with the host–container comparison allowing small numerical slack by design. External validity reaches one dataset and one host–container pair; Azure ran only as a declaration, and the language-model route stayed off.

5 Conclusions

Tied together as Infrastructure-as-Code, the workflow turned a scattered sequence of steps (adaptation, temporal evaluation, persistence, evidence, containers, cloud declaration, and reporting) into one execution that can be re-run and audited. Following design-science guidance on transparent and communicable artifacts [15, 16], the evidence contract is the core of that artifact: ingestion finished in 16.44 s for the evaluated workbook, and the payoff appeared as consistency, since host and container agreed on every non-numeric field and, on the 45 numeric differences, stayed within the declared tolerance even though their full hashes differed.

On the forecasting side, the tie between Naive and SES ($S_1 = 22.28$) and the parsimony rule that resolved it matter less as a performance result than as proof that model selection, forecast, and interval were recorded and reproduced from the same evidence, extending container- and provenance-based reproducibility [11, 12] with an explicit, auditable contract. Read against governance frameworks for AI pipelines [26], the clear next steps are to execute and measure the Azure deployment in full and to replicate the workflow on an independent

domain, adding the continuous-training and drift monitoring this version leaves out.

References

1. Abdalilah Alhalangy. Operationalizing accountable ai through traceable governance architecture for institutional decision support. *Information*, 17(7):694, 2026. <https://doi.org/10.3390/info17070694>.
2. Harald Semmelrock, Tony Ross-Hellauer, Simone Kopeinik, Dieter Theiler, Armin Haberl, Stefan Thalmann, and Dominik Kowald. Reproducibility in machine-learning-based research: Overview, barriers, and drivers. *AI Magazine*, 46(2):e70002, 2025. <https://doi.org/10.1002/aaai.70002>.
3. Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. <https://doi.org/10.1109/ACCESS.2023.3262138>.
4. Josu Diaz-de Arcaya, Ana I. Torre-Bastida, Gorka Zárate, Raúl Miñón, and Aitor Almeida. A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey. *ACM Computing Surveys*, 56(4):1–30, 2024. <https://doi.org/10.1145/3625289>.
5. Beyza Eken, Samodha Pallewatta, Nguyen Khoi Tran, Ayse Tosun, and Muhammad Ali Babar. A multivocal review of MLOps practices, challenges and open issues. *ACM Computing Surveys*, 58(2):1–35, 2025. <https://doi.org/10.1145/3747346>.
6. Firas Bayram and Bestoun S. Ahmed. Towards trustworthy machine learning in production: An overview of the robustness in MLOps approach. *ACM Computing Surveys*, 57(5):1–35, 2025. <https://doi.org/10.1145/3708497>.
7. Matteo Testi, Matteo Ballabio, Emanuele Frontoni, Giulio Iannello, Sara Moccia, Paolo Soda, and Gennaro Vessio. MLOps: A taxonomy and a methodology. *IEEE Access*, 10:63606–63618, 2022. <https://doi.org/10.1109/ACCESS.2022.3181730>.
8. Faezeh Amou Najafabadi, Justus Bogner, Ilias Gerostathopoulos, and Patricia Lago. An architectural perspective on MLOps: Structures, processes, tools, and stakeholders. *Information and Software Technology*, 193:108029, 2026. <https://doi.org/10.1016/j.infsof.2026.108029>.
9. Harald Foidl, Valentina Golendukhina, Rudolf Ramler, and Michael Felderer. Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software*, 207:111855, 2024. <https://doi.org/10.1016/j.jss.2023.111855>.
10. Daochen Zha, Zaid Pervaiz Bhat, Kwei-Herng Lai, Fan Yang, Zhimeng Jiang, Shaochen Zhong, and Xia Hu. Data-centric artificial intelligence: A survey. *ACM Computing Surveys*, 57(5):1–42, 2025. <https://doi.org/10.1145/3711118>.
11. David Moreau, Kristina Wiebels, and Carl Boettiger. Containers for computational reproducibility. *Nature Reviews Methods Primers*, 3:50, 2023. <https://doi.org/10.1038/s43586-023-00236-9>.
12. Sebastiaan P. Huber. Automated reproducible workflows and data provenance with AiiDA. *Nature Reviews Physics*, 4:431, 2022. <https://doi.org/10.1038/s42254-022-00463-1>.
13. Marius Schlegel and Kai-Uwe Sattler. Capturing end-to-end provenance for machine learning pipelines. *Information Systems*, 132:102495, 2025. <https://doi.org/10.1016/j.is.2024.102495>.

14. Lázaro Costa, Susana Barbosa, and Jácome Cunha. A framework for supporting the reproducibility of computational experiments in multiple scientific domains. *Future Generation Computer Systems*, 174:107924, 2026. <https://doi.org/10.1016/j.future.2025.107924>.
15. Petrus M. J. Delpont, Rossouw von Solms, and Mariana Gerber. Methodological guidelines for design science research. *Procedia Computer Science*, 237:195–203, 2024. <https://doi.org/10.1016/j.procs.2024.05.096>.
16. Alan R. Hevner, Jeffrey Parsons, Alfred B. Brendel, Roman Lukyanenko, Verena Tiefenbeck, Monica Chiarini Tremblay, and Jan vom Brocke. Transparency in design science research. *Decision Support Systems*, 182:114236, 2024. <https://doi.org/10.1016/j.dss.2024.114236>.
17. G. Ramesh, T. Vaikunta Pai, Ramona Birau, Karthik K. Poojary, Abhay, Aishwarya R. Shingad, N. Sowjanya, Virgil Popescu, Adrian T. Mitroi, Roxana M. Nioata, and K. M. Kiran Raj. A comprehensive review on scaling machine learning workflows using cloud technologies and DevOps. *IEEE Access*, 13:148559–148594, 2025. <https://doi.org/10.1109/ACCESS.2025.3599281>.
18. Lisana Berberi, Valentin Kozlov, Giang Nguyen, Judith Sáinz-Pardo Díaz, Amanda Calatrava, Germán Moltó, Viet Tran, and Álvaro López García. Machine learning operations landscape: Platforms and tools. *Artificial Intelligence Review*, 58:167, 2025. <https://doi.org/10.1007/s10462-025-11164-3>.
19. SENESCYT. Base de datos de artículos publicados uep en revistas indexadas — conjunto de datos — datos abiertos ecuador. <https://datosabiertos.gob.ec/dataset/base-de-datos-de-articulos-publicados-uep-en-revistas-indexadas>, 2025. Accessed on 10 January 2026.
20. Adrian P. Woźniak, Mateusz Milczarek, and Joanna Woźniak. MLOps components, tools, process, and metrics: A systematic literature review. *IEEE Access*, 13:22166–22175, 2025. <https://doi.org/10.1109/ACCESS.2025.3534990>.
21. Ai Deng. Time series cross validation: A theoretical result and finite sample performance. *Economics Letters*, 233:111369, 2023. <https://doi.org/10.1016/j.econlet.2023.111369>.
22. Ayan Chatterjee, Bestoun S. Ahmed, Erik Hallin, and Anton Engman. Testing of machine learning models with limited samples: An industrial vacuum pumping application. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1280–1290, Singapore, 2022. <https://doi.org/10.1145/3540250.3558943>.
23. Elise Han, Chengpiao Huang, and Kaizheng Wang. Model assessment and selection under temporal distribution shift. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 17374–17392, 2024. <https://proceedings.mlr.press/v235/han24b.html>.
24. Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. <https://doi.org/10.1561/22000000101>.
25. Roberto I. Oliveira, Paulo Orenstein, Thiago Ramos, and João V. Romano. Split conformal prediction and non-exchangeable data. *Journal of Machine Learning Research*, 25:1–38, 2024. Article 225.
26. Talal Ashraf Butt, Muhammad Iqbal, and Noor Arshad. From policy to pipeline: A governance framework for ai development and operations pipelines. *IEEE Access*, 14:1373–1397, 2026. <https://doi.org/10.1109/ACCESS.2025.3647479>.